

UNIT-II

8086 Programming

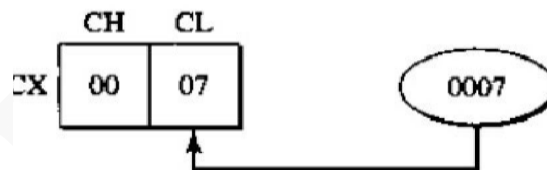
ADDRESSING MODES OF 8086:

Addressing modes indicates way of locating data or operands. Depending upon the data types used in the instruction and the memory addressing modes, any instruction may belong to one or more addressing modes. Thus the addressing modes describe the types of operands and the way they are accessed for executing an instruction.

According to the flow of instruction execution, the instruction may be categorized as: Sequential Control flow instructions & Control Transfer instructions

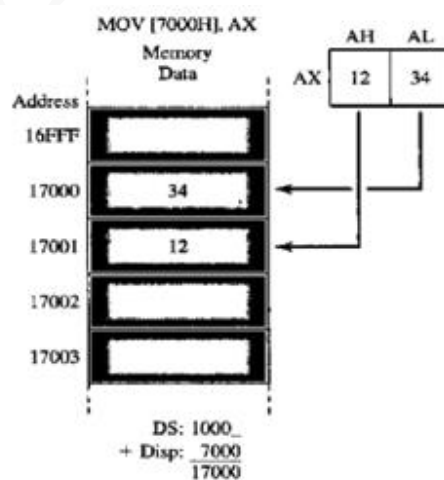
- **Sequential Control flow instructions:** In this type of instruction after execution control can be
 - transferred to the next immediately appearing instruction in the program.
 - The addressing modes for sequential control transfer instructions are as follows:
- **Immediate addressing mode:** In this mode, immediate is a part of instruction and appears in the form of
 - successive byte or bytes.

Example: MOV CX, 0007H; Here 0007 is the immediate data



Direct Addressing mode: In this mode, the instruction operand specifies the memory address where data is located.

Example: MOV AX, [7000H]; Data is available in 7000H memory location
 Effective Address (EA) is computed using 5000H as offset address and content of DS as segment address.
 $EA = 10H * DS + 7000H$



Register Addressing mode: In this mode, the data is stored in a register and it is referred using particular register. All the registers except IP may be used in this mode.

Example: MOV AX, BX;

• **Register Indirect addressing mode:** In this mode, instruction specifies a register containing an address, where data is located. This addressing mode works with SI, DI, BX and BP registers. Example: MOV AX, [BX];

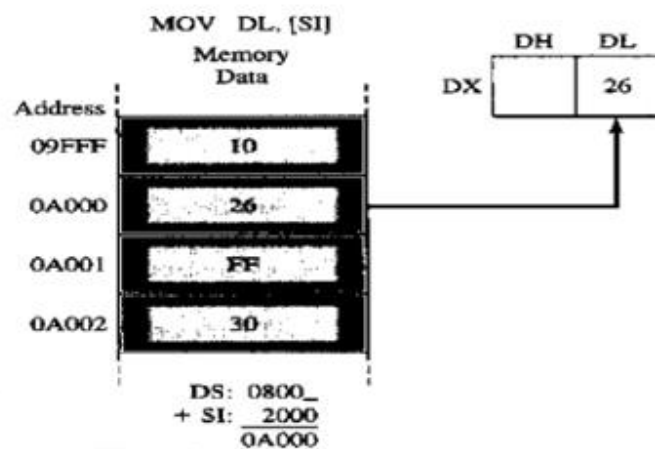
$EA = 10H * DS + [BX]$

• **Indexed Addressing mode:** 8-bit or 16-bit instruction operand is added to the contents of an index register (SI or DI), the resulting value is a pointer to location where data resides. DS and ES are default segments for index registers SI and DI.

DS=0800H, SI=2000H,

MOV DL, [SI]

Example: MOV AX, [SI]; $EA = 10H * DS + [SI]$

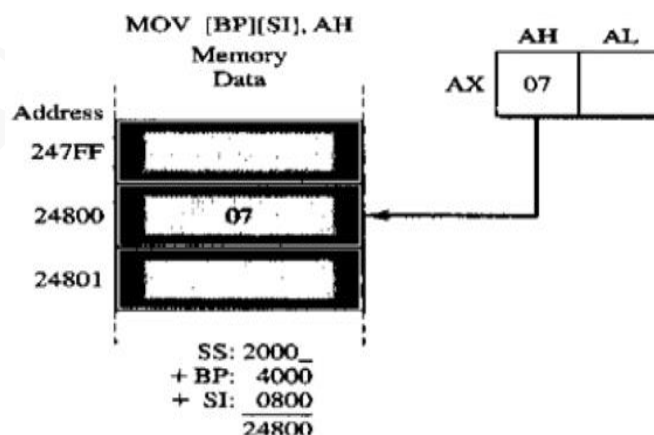


Register Relative Addressing mode: In this mode, the data is available at an effective address formed by adding an 8-bit or 16-bit displacement with the content of any one of the registers BX, BP, SI, DI in the default segments.

Example: MOV AX, 50H [BX]; $EA = 10H * DS + 50H + [BX]$

Based Indexed Addressing mode: In this mode, the contents of a base register (BX or BP) is added to the contents of an index register (SI or DI), the resulting value is a pointer to location where data resides.

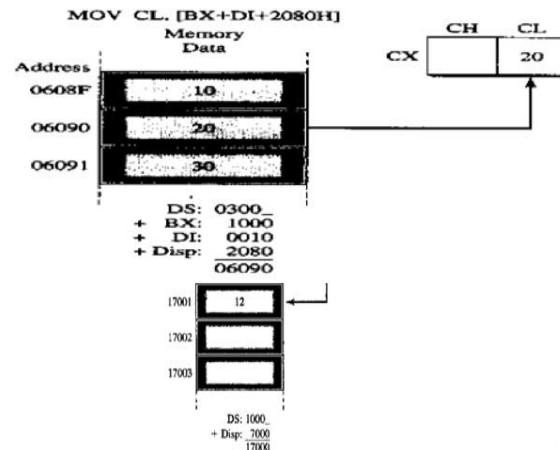
Example: MOV [BP][SI], AX, $EA = 10H * DS + [BP] + [SI]$



Relative Based Indexed Addressing mode: In this mode, 8-bit or 16-bit instruction operand is added to the contents of a base register (BX or BP) and index register (SI or DI), the resulting value is a pointer to location where data resides.

Example: MOV AX, 50H [BX] [SI];

$EA = 10H * DS + 50H + [BX] + [SI]$



• **Control Transfer Instructions:** In control transfer instruction, the control can be transferred to some predefined address or the address somehow specified in the instruction after their execution. For the control transfer instructions, the addressing modes depend upon whether the destination location is within the segment or different segments. It also depends upon the method of passing the destination address to the processor. Depending on this control transfer instructions are categorized as follows:

Intra Segment Direct Mode

If the location to which control is to be transferred is in the same segment, it is called intra segment mode. If address to which the control is to be transferred appears directly in the instruction as a displacement value, it is what called as intra segment direct mode. For example,

JMP SHORT LABEL;

is a control transfer instruction following intra segment direct mode. Here, SHORT LABEL represents a signed displacement.

Intra Segment Indirect Mode

If the location to which control is to be transferred is in the same segment, it is called intra segment mode. If address to which the control is to be transferred appears indirectly in the instruction, it is what called as intra segment indirect mode. For example,

JMP [AX]; is a control transfer instruction following intra segment indirect mode.

Inter Segment Direct Mode

If the location to which control is to be transferred is not in the same segment, it is called inter segment mode. If address of segment to which the control is to be transferred and location in the segment appears directly in the instruction, it is what called as inter segment direct mode. For example,

JMP 2000H : 3000H;

is a control transfer instruction following inter segment direct mode. Here, Jump is to effective address 3000H in segment at 2000H.

Inter Segment Indirect Mode

If the location to which control is to be transferred is not in the same segment, it is called inter segment mode. If address of segment to which the control is to be transferred and location in the segment appears indirectly in the instruction, it is what called as inter segment indirect mode. For example,

JMP [5000H];

is a control transfer instruction following inter segment indirect mode.

Instruction Set of 8086:

There are 117 basic instructions in the instruction set of 8086. The instruction set of 8086 can be

1.	Data copy / Transfer instructions	2.	Arithmetic and Logical instructions
3.	Branch instructions	4.	Loop instructions
5.	Machine control instructions	6.	Flag Manipulation instructions
7.	Shift and Rotate instructions	8.	String instructions

Data copy / Transfer instructions: The data movement instructions copy values from one location to another.

These instructions include **MOV, XCHG, LDS, LEA, LES, PUSH, PUSHF,**

PUSHFD, POP, POPF, LAHF, AND SAHF.

MOV The MOV instruction copies a word or a byte of data from source to a destination. The destination can be a register or a memory location. The source can be a register, or memory location or immediate data. MOV instruction does not affect any flags. The mov instruction takes several different forms:

The MOV instruction cannot:

1. Set the value of the CS and IP registers.
2. Copy value of one segment register to another segment register (should copy to general register first). MOV CS, DS (Invalid)

3. Copy immediate value to segment register (should copy to general register first).

MOV CS, 2000H (Invalid)

Example:

ORG 100h

MOV AX, 0B800h ; set AX = B800h

MOV DS, AX ; copy value of AX to DS.

MOV CL, 'A' ; CL = 41h (ASCII code).

The XCHG Instruction: Exchange This instruction exchanges the contents of the specified source and destination operands, which may be registers or one of them, may be a memory location.

However, exchange of data contents of two memory locations is not permitted.

Example: MOV AL, 5 ; AL = 5

MOV BL, 2 ; BL = 2

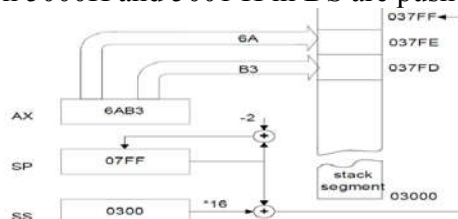
XCHG AL, BL ; AL = 2, BL = 5

PUSH: Push to stack; this instruction pushes the contents of the specified register/memory location on to the stack. The stack pointer is decremented by 2, after each execution of the instruction. The actual current stack top is always occupied by the previously pushed data. Hence, the push operation decrements SP by two and then stores the two byte contents of the operand onto the stack. The higher byte is pushed first and then the lower byte. Thus out of the two decremented stack addresses the higher byte occupies the higher address and the lower byte occupies the lower address.

1. PUSH AX

2. PUSH DS

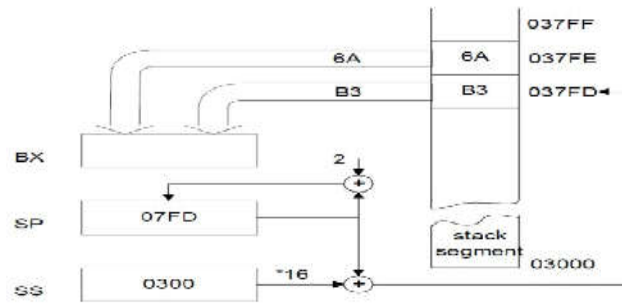
3. PUSH [5000H] ; Content of location 5000H and 5001 H in DS are pushed onto the stack



The effect of PUSH AX instruction

POP: Pop from Stack this instruction when executed loads the specified register/memory location with the contents of the memory location of which the address is formed using the current stack segment and stack pointer as usual. The stack pointer is incremented by 2. The POP instruction serves exactly opposite to the PUSH instruction.

1. POP BX
2. POP DS
3. POP [5000H]



The effect of POP BX instruction

PUSHF: Push Flags to Stack The push flag instruction pushes the flag register on to the stack; first the upper byte and then the lower byte will be pushed on to the stack. The SP is decremented by 2, for each push operation. The general operation of this instruction is similar to the PUSH operation.

POPF: Pop Flags from Stack The pop flags instruction loads the flag register completely (both bytes) from the word contents of the memory location currently addressed by SP and SS. The SP is incremented by 2 for each pop operation.

LAHF: Load AH from Lower Byte of Flag This instruction loads the AH register with the lower byte of the flag register. This instruction may be used to observe the status of all the condition code flags (except overflow) at a time.

SAHF: Store AH to Lower Byte of Flag Register This instruction sets or resets the condition code flags (except overflow) in the lower byte of the flag register depending upon the corresponding bit positions in AH. If a bit in AH is 1, the flag corresponding to the bit position is set, else it is reset.

LEA: Load Effective Address The load effective address instruction loads the offset of an operand in the specified register. This instruction is similar to MOV, MOV is faster than LEA.

LEA cx, [bx+si] ; CX (BX+SI) mod 64K If bx=2f00 H; si=10d0H cx 3fd0H

The LDS AND LES instructions:

- LDS and LES load a 16-bit register with offset address retrieved from a memory location then load either DS or ES with a segment address retrieved from memory.

This instruction transfers the 32-bit number, addressed by DI in the data segment, into the BX and DS registers.

- LDS and LES instructions obtain a new far address from memory.

– offset address appears first, followed by the segment address

- This format is used for storing all 32-bit memory addresses.

- A far address can be stored in memory by the assembler.

LDS BX,DWORD PTR[SI] BL

[SI];

BH [SI+1]

DS [SI+3:SI+2]; in the data segment

LES BX,DWORD PTR[SI]
 BL [SI];
 BH [SI+1]
 ES [SI+3:SI+2]; in the extra segment

IN – IN Accumulator, Port

The IN instruction copies data from a port to the AL or AX register. If an 8-bit port is read, the data will go to AL. If a 16-bit port is read, the data will go to AX.

IN AX, 34H Input a word from port 34H to AX

For the variable-port form of the IN instruction, the port address is loaded into the DX register before the IN instruction. Since DX is a 16-bit register, the port address can be any number between 0000H and FFFFH. Therefore, up to 65,536 ports are addressable in this mode.

MOV DX, 0FF78H Input a byte from 8-bit port 0FF78H to AL

IN AL, DX Initialize DX to point to port

OUT – OUT Port, Accumulator

The OUT instruction copies a byte from AL or a word from AX to the specified port. The OUT instruction has two possible forms, fixed port and variable port. For the fixed port form, the 8-bit port address is specified directly in the instruction. With this form, any one of 256 possible ports can be addressed.

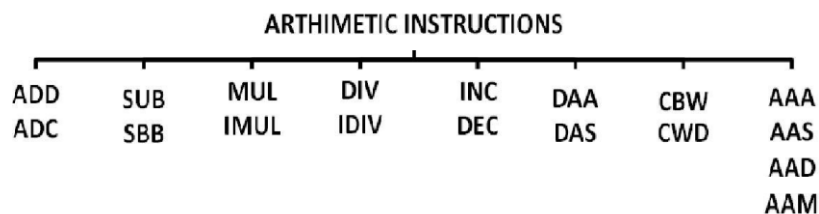
OUT 3BH, AL Copy the content of AL to port 3BH

For variable port form of the OUT instruction, the content of AL or AX will be copied to the port at an address contained in DX. Therefore, the DX register must be loaded with the desired port address before this form of the OUT instruction is used.

MOV DX, 0FFF8H Copy content of AL to port FFF8H

OUT DX, AL Load desired port address in DX

Arithmetic instruction



ADD – ADD Destination, Source

ADC – ADC Destination, Source

These instructions add a number from some *source* to a number in some *destination* and put the result in the specified destination. The ADC also adds the status of the carry flag to the result. The source may be an immediate number, a register, or a memory location.

ADD AL, 74H Add immediate number 74H to content of AL. Result in AL

ADC CL, BL Add content of BL plus carry status to content of CL

ADD DX, [SI] Add word from memory at offset [SI] in DS to content of DX

SUB – SUB Destination, Source

SBB – SBB Destination, Source

These instructions subtract the number in some *source* from the number in some *destination* and put the result in the destination. The SBB instruction also subtracts the content of carry flag from the destination. The source may be an immediate number, a register or memory location.

SUB CX, BX CX – BX; Result in CX
SUB AX, 3427H Subtract immediate number 3427H from AX

MUL – MUL Source

This instruction multiplies an *unsigned* byte in some *source* with an *unsigned* byte in AL register or an unsigned word in some *source* with an unsigned word in AX register. When a byte is multiplied by the content of AL, the result (product) is put in AX. When a word is multiplied by the content of AX, the result is put in DX and AX registers.

MUL BL Multiply AL with BL; result in AX
MUL CX Multiply AX with CX; result high word in DX, low word in AX

IMUL – IMUL Source

This instruction multiplies a *signed* byte from *source* with a *signed* byte in AL or a *signed* word from some *source* with a *signed* word in AX. When a byte from source is multiplied with content of AL, the signed result (product) will be put in AX. When a word from source is multiplied by AX, the result is put in DX and AX.

IMUL BL Multiply signed byte in AL with signed byte in BL; result in AX.
IMUL BX Multiply BX with AX; result in DX and AX

DIV – DIV Source

This instruction is used to divide an *unsigned* word by a byte or to divide an *unsigned* double word (32 bits) by a word. When a word is divided by a byte, the word must be in the AX register. The divisor can be in a register or a memory location. After the division, AL will contain the **8-bit quotient**, and AH will contain the **8-bit remainder**.

When a double word is divided by a word, the most significant word of the double word must be in DX, and the least significant word of the double word must be in AX. After the division, AX will contain the **16-bit quotient** and DX will contain the **16-bit remainder**.

DIV BL Divide word in AX by byte in BL; Quotient in AL, remainder in AH
DIV CX Divide the word in DX and AX by word in CX; Quotient in AX, and remainder in DX

IDIV – IDIV Source

This instruction is used to divide a *signed* word by a *signed* byte, or to divide a *signed* double word by a *signed* word. When dividing a signed word by a signed byte, the word must be in the AX register. The divisor can be in an 8-bit register or a memory location. After the division, AL will contain the signed quotient, and AH will contain the signed remainder. When dividing a signed double word by a signed word, the most significant word of the dividend (numerator) must be in the DX register, and the least significant word of the dividend must be in the AX register. The divisor can be in any other 16-bit register or memory location. After the division, AX will contain a signed 16-bit quotient, and DX will contain a signed 16-bit remainder.

IDIV BL Signed word in AX/signed byte in BL
IDIV BP Signed double word in DX and AX/signed word in BP

INC – INC Destination

The INC instruction adds 1 to a specified register or to a memory location..

INC BL Add 1 to content of BL register

DEC – DEC Destination

This instruction subtracts 1 from the destination word or byte.

DEC CL Subtract 1 from content of CL register

CBW (CONVERT SIGNED BYTE TO SIGNED WORD)

This instruction copies the sign bit of the byte in AL to all the bits in AH. AH is then said to be the sign extension of AL

Let AX = 00000000 10011011 (–155 decimal)

OUTPUT : AX = 11111111 10011011 (–155 decimal)

CWD (CONVERT SIGNED WORD TO SIGNED DOUBLE WORD)

This instruction copies the sign bit of a word in AX to all the bits of the DX register. In other words, it extends the sign of AX into all of DX.

AX = 11110000 11000111 (–3897 decimal)

OUTPUT : DX = 11111111 11111111 AX = 11110000 11000111 (–3897 decimal)

LOGICAL INSTRUCTIONS

AND – AND Destination, Source

This instruction ANDs each bit in a source byte or word with the same numbered bit in a destination byte or word. The result is put in the specified destination.

AND BH, CL

AND BX, 00FFH

OR – OR Destination, Source

This instruction ORs each bit in a source byte or word with the same numbered bit in a destination byte or word. The result is put in the specified destination.

OR AH, CL CL ORed with AH, result in AH, CL not changed

OR BL, 80H BL ORed with immediate number 80H; sets MSB of BL to 1

XOR – XOR Destination, Source

This instruction Exclusive-ORs each bit in a source byte or word with the same numbered bit in a destination byte or word. The result is put in the specified destination.

XOR CL, BH

XOR BP, DI

NOT – NOT Destination

The NOT instruction inverts each bit (forms the 1's complement) of a byte or word in the specified destination.

NOT BX Complement content of BX register

NEG – NEG Destination

This instruction replaces the number in a destination with its 2's complement. It gives the same result as the *invert each bit and add one* algorithm.

NEG AL Replace number in AL with its 2's complement

CMP – CMP Destination, Source

This instruction compares a byte / word in the specified source with a byte / word in the specified destination. The comparison is actually done by **subtracting** the source byte or word from the destination byte or word. The source and the destination are not changed, but the flags are set to indicate the results of the comparison.

CMP CX,BX

	CF	ZF	SF	
CX = BX	0	1	0	Result of subtraction is 0
CX > BX	0	0	0	No borrow required, so CF = 0
CX < BX	1	0	1	Subtraction requires borrow, so CF = 1

TEST – TEST Destination, Source

This instruction ANDs the byte / word in the specified source with the byte / word in the specified destination. **Flags are updated, but neither operand is changed.** The test instruction is often used to set flags before a Conditional jump instruction.

TEST AL, BH AND BH with AL. No result stored; Update PF, SF, ZF

ROTATE AND SHIFT INSTRUCTIONS

RCL – RCL Destination, Count

This instruction rotates all the bits in a specified word or byte some number of bit positions to the left. The operation is circular because the MSB of the operand is rotated into the carry flag and the bit in the carry flag is rotated around into LSB of the operand



To rotate the operand by one bit position, specify this by putting a 1 in the count position of the instruction. To rotate by more than one bit position, load the desired number into the CL register and put “CL” in the count position of the instruction.

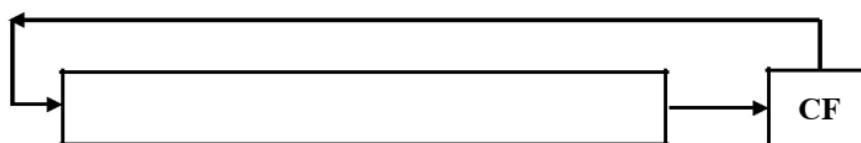
RCL DX, 1 Word in DX 1 bit left

MOV CL, 4 Load the number of bit positions to rotate into CL

RCL DX, CL Rotate DX register content 4 times left

RCR – RCR Destination, Count

This instruction rotates all the bits in a specified word or byte some number of bit positions to the right. The operation is circular because the LSB of the operand is rotated into the carry flag and the bit in the carry flag is rotated around into MSB of the operand.



If you want to rotate the operand by one bit position, you can specify this by putting a 1 in the count position of the instruction. To rotate more than one bit position, load the desired number into the CL register and put “CL” in the count position of the instruction.

RCR BX, 1	Word in BX right 1 bit
MOV CL, 4	Load CL for rotating 4 bit position
RCRBX, CL	Rotate BX register content 4 times right

ROL – **ROL** **Destination,** **Count**
This instruction rotates all the bits in a specified word or byte to the left some number of bit positions. The data bit rotated out of MSB is circled back into the LSB. It is also copied into CF.

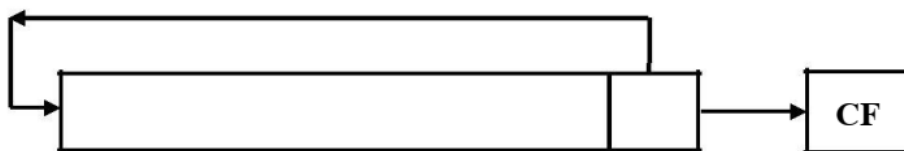


If you to want rotate the operand by one bit position, you can specify this by putting 1 in the count position in the instruction. To rotate more than one bit position, load the desired number into the CL register and put “CL” in the count position of the instruction.

ROL AX, 1	Rotate the word in AX 1 bit position left
MOV CL, 04H	Load number of bits to rotate in CL
ROL BL, CL	Rotate BL register content 4 times left

ROR – **ROR** **Destination, Count**

This instruction rotates all the bits in a specified word or byte some number of bit positions to right. The operation is desired as a rotate rather than shift, because the bit moved out of the LSB is rotated around into the MSB. The data bit moved out of the LSB is also copied into CF.



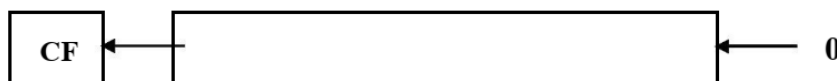
To rotate the operand by one bit position, specify this by putting 1 in the count position in the instruction. To rotate by more than one bit position, load the desired number into the CL register and put “CL” in the count position of the instruction.

ROR BL, 1 Rotate all bits in BL right 1 bit position

SAL – **SAL** **Destination, Count**

SHL – **SHL** **Destination, Count**

This instruction shifts each bit in the specified destination some number of bit positions to the left. As a bit is shifted out of the LSB operation, a 0 is put in the LSB position. The MSB will be shifted into CF.



To shift the operand by one bit position, specify this by putting a 1 in the count position of the instruction. For shifts of more than 1 bit position, load the desired number of shifts into the CL register, and put “CL” in the count position of the instruction.

SAL BX, 1	Shift word in BX 1 bit position left, 0 in LSB
MOV CL, 02H	Load desired number of shifts in CL
SAL BX, CL	Shift word in BX left CL bit positions, 0 in LSBs

SAR – SAR Destination, Count

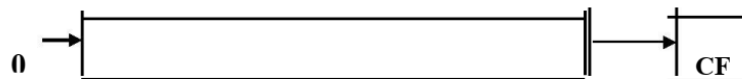
This instruction shifts each bit in the specified destination some number of bit positions to the right. As a bit is shifted out of the MSB position, a copy of the old MSB is put in the MSB position. In other words, the sign bit is copied into the MSB. The LSB will be shifted into CF



To shift the operand by one bit position, specify this by putting a 1 in the count position of the instruction. For shifts of more than 1 bit position, load the desired number of shifts into the CL register, and put “CL” in the count position of the instruction. SAR DX, 1 Shift word in DX one bit position right, new MSB = old MSB

SHR – SHR Destination, Count

This instruction shifts each bit in the specified destination some number of bit positions to the right. As a bit is shifted out of the MSB position, a 0 is put in its place. The bit shifted out of the LSB position goes to CF.



To shift the operand by one bit position, specify this by putting a 1 in the count position of the instruction. For shifts of more than 1 bit position, load the desired number of shifts into the CL register, and put “CL” in the count position of the instruction.

SHR BP, 1 Shift word in BP one bit position right, 0 in

MOV CL, 03H MSB Load desired number of shifts into CL

SHR BP, CL Shift BP register content 3 bits right; 0's in 3 MSBs

CONDITIONAL TRANSFER INSTRUCTIONS:

Conditional Jump: The conditional jump instructions are the basic tool for creating loops and other conditionally executable statements like if....then statement. The conditional jumps test one or more bits in the status register to see if they match some particular pattern. If the pattern matches, control transfers to the target location. If the condition fails, the CPU ignores the conditional jump and execution continues with the next instruction. Some instructions, for example, test the conditions of the sign, carry, overflow and zero flags.

Instruction Code	Description	Condition for jump
JA/JNBE	Jump if above/Jump if not below or equal.	CF = 0 and ZF = 0
JAЕ/JNB	Jump if above or equal/Jump if not below.	CF = 0 and ZF = 1
JB/JNAE/JC	Jump if below/Jump if not above or equal.	CF = 1 and ZF = 0
JBE/JNA	Jump if below or equal/Jump if not above.	CF = 1 and ZF = 1
JE/JZ	Jump if equal/Jump if zero flag.	ZF = 1
JG/JNLE	Jump if greater/Jump if not less than nor equal.	ZF = 0 and CF = 0
JGE/JNL	Jump if greater than or equal/Jump if not less than.	SF = 0
JL/JNGE	Jump if less than/Jump if not greater than or equal.	SF ≠ 0
JLE/JNG	Jump if less than or equal/Jump if not greater	ZF = 1 or SF ≠ 0
JNC	Jump if no carry	CF = 0
JNE/JNZ	Jump if not equal/Jump if not zero	ZF = 0
JNO	Jump if no overflow	OF = 0
JNP/JPO	Jump if not parity/Jump if parity odd	PF = 0
JNS	Jump if not sign or jump if positive	SF = 0
JO	Jump if overflow flag = 1.	OF = 1
JP/JPE	Jump if parity/Jump if parity even.	PF = 1
JS	Jump if sign flag = 1 or jump if negative	SF = 1
JCXZ	Jump if CX is zero	CX = 0

Iteration Control (LOOP) Instructions:

These instructions are used to execute a series of instructions some number of times. The number is specified in the CX register. The CX register is automatically decremented by one, each time after execution of LOOP instruction. Until CX = 0, execution will jump to a destination specified by a label in the instruction. The destination address for the jump must be in the range of – 128 bytes to + 127 bytes from the address of the instruction after the iteration control instruction. For LOOPE/LOOPZ and LOOPNE/LOOPNZ instructions there is one more condition for exit from loop, which is given below. If the loop is not taken, execution simply goes on to the next instruction after the iteration control instruction.

Instruction Code	Description	Condition for Exit
1. LOOP	Loop through a sequence of instructions	CX = 0
2. LOOPE/LOOPZ	Loop through a sequence of instructions	CX = 0 or ZF = 0
3. LOOPNE/LOOPNZ	Loop through a sequence of instructions	CX = 0 or ZF = 1

UNCONDITIONAL TRANSFER INSTRUCTIONS:

IRET (INTERRUPT RETURN)

When the 8086 responds to an interrupt signal or to an interrupt instruction, it pushes the flags, the current value of CS, and the current value of IP onto the stack. It then loads CS and IP with the starting address of the procedure, which you write for the response to that interrupt. The IRET instruction is used at the end of the interrupt service procedure to return execution to the interrupted program

INT N – INT TYPE N

The term *type* in the instruction format refers to a number between 0 and 255, which identify the interrupt.

JMP (UNCONDITIONAL JUMP TO SPECIFIED DESTINATION)

This instruction will fetch the next instruction from the location specified in the instruction rather than from the next location after the JMP instruction. Two types of Jump instruction. Far Jump and Near Jump. If the destination is in the same code segment as the JMP instruction, then only the instruction pointer will be changed to get the destination location. This is referred to as a **near jump**. If the destination for the jump instruction is in a segment with a name different from that of the segment containing the JMP instruction, then both the instruction pointer and the code segment register content will be changed to get the destination location. This referred to as a **far jump**. The JMP instruction does not affect any flag

RET Instruction : The RET instruction will return 8086 Program Execution Transfer Instructions from a procedure to the next instruction after the CALL instruction in the calling program. If the procedure is a near procedure (in the same code segment as the CALL instruction), then the return will be done by replacing the instruction pointer with a word from the top of the stack.

CALL Instruction : The CALL instruction is used to 8086 Program Execution Transfer Instructions to a subprogram or procedure. There are two basic types of CALLs, near and far. A near CALL is a call to a procedure which is in the same code segment as the CALL instruction. When the 8086 executes a near CALL instruction it decrements the stack pointer by two and copies the offset of the next instruction after the CALL on the stack. It loads IP with the offset of the first instruction of the procedure in same segment. The near CALL is also known as intrasegment CALL

MACHINE/PROCESS CONTROL INSTRUCTIONS

The Machine control instructions control the bus usage and execution

WAIT – Wait for Test input pin to go low.

HLT – Halt the process.

NOP – No operation.

ESC – Escape to external device like NDP

LOCK – Bus lock instruction prefix.

HLT (HALT PROCESSING)

The HLT instruction causes the 8086 to stop fetching and executing instructions. The 8086 will enter a halt state. The different ways to get the processor out of the halt state are with an interrupt signal on the INTR pin, an interrupt signal on the NMI pin, or a reset signal on the RESET input.

NOP (PERFORM NO OPERATION)

This instruction simply uses up three clock cycles and increments the instruction pointer to point to the next instruction. The NOP instruction can be used to increase the delay of a delay loop. When hand coding, a NOP can also be used to hold a place in a program for an instruction that will be added later. NOP does not affect any flag.

ESC (ESCAPE)

This instruction is used to pass instructions to a coprocessor, such as the 8087 Math coprocessor, which shares the address and data bus with 8086. Instructions for the coprocessor are represented by a 6-bit code embedded in the ESC instruction. As the 8086 fetches instruction bytes, the coprocessor also fetches these bytes from the data bus and puts them in its queue

LOCK – ASSERT BUS LOCK SIGNAL

Many microcomputer systems contain several microprocessors. Each microprocessor has its own local buses and memory. Each microprocessor takes control of the system bus only when it needs to access some system resources. The LOCK prefix allows a microprocessor to make sure that another processor does not take control of the system bus while it is in the middle of a critical instruction, which uses the system bus

WAIT – WAIT FOR SIGNAL OR INTERRUPT SIGNAL

When this instruction is executed, the 8086 enters an idle condition in which it is doing no processing. The 8086 will stay in this idle state until the 8086 test input pin is made low or until an interrupt signal is received on the INTR or the NMI interrupt input pins. If a valid interrupt occurs while the 8086 is in this idle state, the 8086 will return to the idle state after the interrupt service procedure executes. It returns to the idle state because the address of the WAIT instruction is the address pushed on the stack when the 8086 responds to the interrupt request.

STRING MANIPULATION INSTRUCTIONS

MOVS / MOVSB / MOVSW

This instruction copies a byte or a word from location in the data segment to a location in the extra segment. The offset of the source in the data segment must be in the SI register. The offset of the destination in the extra segment must be in the DI register. For multiple-byte or multiple-word moves, the number of elements to be moved is put in the CX register so that it can function as a counter. After the byte or a word is moved, SI and DI are automatically adjusted to point to the next source element and the next destination element. If DF is 0, then SI and DI will be incremented by 1 after a byte move and by 2 after a word move. If DF is 1, then SI and DI will be decremented by 1 after a byte move and by 2 after a word move

```
MOV SI, 5000
MOV DI, 6000
CLD
MOV CX, 04H
REP MOVSB
```

LODS / LODSB / LODSW (LOAD STRING BYTE INTO AL OR STRING WORD INTO AX)

This instruction copies a byte from a string location pointed to by SI to AL, or a word from a string location pointed to by SI to AX. If DF is 0, SI will be automatically incremented (by 1 for a byte string, and 2 for a word string) to point to the next element of the string. If DF is 1, SI will be automatically decremented (by 1 for a byte string, and 2 for a word string) to point to the previous element of the string. LODS does not affect any flag.

STOS / STOSB / STOSW (STORE STRING BYTE OR STRING WORD)

This instruction copies a byte from AL or a word from AX to a memory location in the extra segment pointed to by DI. In effect, it replaces a string element with a byte from AL or a word from AX. After the copy, DI is automatically incremented or decremented to point to next or previous element of the string.

CMPS / CMPSB / CMPSW (COMPARE STRING BYTES OR STRING WORDS)

This instruction can be used to compare a byte / word in one string with a byte / word in another string. SI is used to hold the offset of the byte or word in the source string, and DI is used to hold the offset of the byte or word in the destination string. The AF, CF, OF, PF, SF, and ZF flags are affected by the comparison, but the two operands are not affected.

SCAS / SCASB / SCASW (SCAN A STRING BYTE OR A STRING WORD)

SCAS compares a byte in AL or a word in AX with a byte or a word in ES pointed to by DI. Therefore, the string to be scanned must be in the extra segment, and DI must contain the offset of the byte or the word to be compared. If DF is cleared, then DI will be incremented by 1 for byte strings and by 2 for word strings. If DF is set, then DI will be decremented by 1 for byte strings and by 2 for word strings. SCAS affects AF, CF, OF, PF, SF, and ZF, but it does not change either the operand in AL (AX) or the operand in the string.

REP / REPE / REPZ / REPNE / REPNZ (PREFIX) (REPEAT STRING INSTRUCTION UNTIL SPECIFIED CONDITIONS EXIST)

REP is a prefix, which is written before one of the string instructions. It will cause the CX register to be decremented and the string instruction to be repeated until CX = 0. The instruction REP MOVSB, for example, will continue to copy string bytes until the number of bytes loaded into CX has been copied.

FLAG MANIPULATION INSTRUCTIONS

STC (SET CARRY FLAG) sets the carry flag to 1.

CLC (CLEAR CARRY FLAG) resets the carry flag to 0.

CMC (COMPLEMENT CARRY FLAG) complements the carry flag.

STD (SET DIRECTION FLAG) sets the direction flag to 1.

CLD (CLEAR DIRECTION FLAG) resets the direction flag to 0

STI (SET INTERRUPT FLAG) Setting the interrupt flag to a 1 enables the INTR interrupt input

CLI (CLEAR INTERRUPT FLAG) resets the interrupt flag to 0.

STACK RELATED INSTRUCTIONS

PUSH – PUSH Source

The PUSH instruction decrements the stack pointer by 2 and copies a word from a specified source to the location in the stack segment to which the stack pointer points.

PUSH BX Decrement SP by 2, copy BX to stack

POP – POP Destination

The POP instruction copies a word from the stack location pointed to by the stack pointer to a destination specified in the instruction. After the word is copied to the specified destination, the stack pointer is automatically incremented by 2 to point to the next word on the stack POP DX Copy a word from top of stack to DX; increment SP by 2

PUSHF (PUSH FLAG REGISTER TO STACK)

The PUSHF instruction decrements the stack pointer by 2 and copies a word in the flag register to two memory locations in stack pointed to by the stack pointer.

POPF (POP WORD FROM TOP OF STACK TO FLAG REGISTER)

The POPF instruction copies a word from two memory locations at the top of the stack to the flag register and increments the stack pointer by 2.

ASSEMBLER DIRECTIVES AND OPERATORS

An assembler is a program used to convert an assembly language program into the equivalent machine code modules which may further be converted to executable codes. The assembler decides the address of each label and substitutes the values for each of the constants and variables. It then forms the machine code for the mnemonics and data in the assembly language program. While doing these things, the assembler may find out syntax errors.

The **DB directive** is used to declare a byte-type variable or to set aside one or more storage locations of type byte in memory (Define Byte)

Example: Temp DB 42H; Temp is a variable allotted 1byte of memory location assigned with data 42H

The **DW directive** is used to declare a variable of type word or to reserve memory locations which can be accessed as type double word (Define word)

Example: N2 DW 427AH; N2 variable is initialized with value 427AH when it is loaded into memory to run.

The **DD directive** is used to declare a variable of type double word or to reserve memory locations which can be accessed as type double word (Define double word)

Example: Big DD 2456756CH; Big variable is initialized with 4 bytes

The **DQ directive** is used to tell the assembler to declare a variable 4 words in length or to reserve 4 words of storage in memory (Define Quadword)

Example: Big DQ 2456756C88464567H; Big variable is initialized with 4 words (8 bytes)

The **DT directive** is used to tell the assembler to declare a variable 10 bytes in length or to reserve 10 bytes of storage in memory (Define Ten bytes)

Example: Packed BCD DT 11223344556677889900H; 10 byte data is initialized to variable packed BCD
DUP: This directive operator is used to initialize several locations and to assign values to these locations.
Its format is: Name Data-Type Num DUP (value)

Example: TABLE DB 20 DUP(0) ; Reserve an array of 20 bytes of memory and initialize all 20 bytes with 0. Array is named TABLE

END: The **END** directive is placed after the last statement of a program to tell the assembler that this is the end of the program module. The assembler will ignore any statement after an end directive.

The **ENDP** directive is used with the name of the procedure to indicate the end of a procedure to the

The **ENDS** directive is used with the name of the segment to indicate the end of a segment to the assembler.

CODE SEGMENT

...

...

CODE ENDS

EQU: The **EQU** directive is used to give a name to some value or to a symbol. Each time assembler finds the name in the program it will replace the name with the value.

FACTOR EQU 03H; This statement should be written at the start

ADD AL, FACTOR; The assembler converts this instruction as ADD AL, 03H

EVEN: The **EVEN** directive instructs the assembler to increment the location of the counter to the next even address if it is not already in the even address. If the word starts at an odd address, 8086 will take 2 bus cycles to get the 2 byte of the word. *"A series of words can read much more quickly if they are at even address"*.

DATA HERE SEGMENT ; Location counter will point to 0009H after assembler reads next statement

SALES DB 9 DUP (?) ;Declare an array of 9 bytes

EVEN ; Increment location counter to 000AH

RECORD DW 100 DUP (?) ; Array of 100 words starting on even address for quicker read

DATA HERE ENDS ;

GLOBAL: This **GLOBAL** directive can be used in place of **PUBLIC** directive or in place of an **EXTRN** directive. The **GLOBAL** directive is used to make the symbol available to other modules.

PUBLIC: The **PUBLIC** directive is used along with the **EXTRN** directive. This informs the assembler that the labels, variables, constants, or procedures declared **PUBLIC** may be accessed by other assembly modules to form their codes, but while using the **PUBLIC** declared labels, variables, constants or procedures the user must declare them externals using the **EXTRN** directive.

EXTRN: This **EXTRN** directive is used to tell the assembler that the names or labels following the directive are in some other assembly module.

Labels: A label, a symbolic name for a particular location in an instruction sequence, maybe defined in one of three ways. The first way is the most common. The format is shown below: **label: [instruction]** where "label" is a unique ASM86 identifier and "instruction" is an 8086/8087/8088 instruction. This label will have the following attributes:

1. Segment-the current segment being assembled.
2. Offset-the current value of the location counter.
3. Type-will be NEAR.

An example of this form of label definition is: ALAB: MOV AX, COUNT

LENGTH: Byte Length of a Label

This is used to refer to the length of a data array or a string.

MOV CX, LENGTH ARRAY

This statement, when assembled, will substitute the length of the array ARRAY in bytes, in the instruction.

LOCAL

The label, variables, constants or procedures declared LOCAL in a module are to be used only by that module.

LOCAL a, b, DATA, ARRAY, ROUTINE

OFFSET: Offset of a Label

When the assembler comes across the OFFSET operator along with a label, it first computes the 16-bit displacement (also called as offset interchangeably) of the particular label, and replaces the string 'OFFSET LABEL' by the computed displacement.

```
CODE SEGMENT
    MOV SI, OFFSET LIST
CODE ENDS

DATA SEGMENT
    LIST DB 10H
DATA ENDS
```

ORG: Origin

The ORG directive directs the assembler to start the memory allotment for the particular segment, block or code from the declared address in the ORG statement. While starting the assembly process for a module, the assembler initializes a location counter to keep track of the allotted addresses for the module. If the ORG statement is not written in the program, the location counter is initialized to 0000. If an ORG 200H statement is present at the starting of the code segment of that module, then the code will start from 200H address in code segment. In other words, the location counter will get initialized to the address 0200H instead of 0000H. Thus, the code for different modules and segments can be located in the available memory as required by the programmer. The ORG directive can even be used with data segments similarly.

PROC: Procedure

The PROC directive marks the start of a named procedure in the statement. Also, the types NEAR or FAR specify the type of the procedure, i.e whether it is to be called by the main program located within 64K of physical memory or not.

RESULT PROC NEAR

ROUTINE PROC FAR

PTR: Pointer

The pointer operator is used to declare the type of a label, variable or memory operand. The operator PTR is prefixed by either BYTE or WORD. If the prefix is BYTE, then the particular label, variable or memory operand is treated as an 8-bit quantity, while if WORD is the prefix, then it is treated as a 16-bit quantity. In other words, the PTR operator is used to specify the data type - byte or word.

MOV AL, BYTE PTR [SI] Moves content of memory location addressed by SI (8-bit) to AL

INC BYTE PTR [BX] Increments byte contents of memory location addressed by BX

SEG: Segment of a Label

The SEG operator is used to decide the segment address of the label, variable, or procedure and substitutes the segment base address in place of 'SEG label'. The example given below explain the use of SEG operator.

MOV AX, SEG ARRAY This statement moves the segment address of ARRAY
MOV DS, AX in which it is appearing, to register AX and then to DS.

SHORT

The SHORT operator indicates to the assembler that only one byte is required to code the displacement for a jump (i.e. displacement is within -128 to +127 bytes from the address of the byte next to the jump opcode).

JMP SHORT LABEL

TYPE

The TYPE operator directs the assembler to decide the data type of the specified label and replaces the 'TYPE label' by the decided data type. For the word type variable, the data type is 2, for double word type, it is 4, and for byte type, it is 1. Suppose, the STRING is a word array. The instruction MOV AX, TYPE STRING moves the value 0002H in AX.

GLOBAL

The labels, variables, constants or procedures declared GLOBAL may be used by other modules of the program. Once a variable is declared GLOBAL, it can be used by any module

SEGMENT: Logical Segment The SEGMENT directive marks the starting of a logical segment.

The start segment is also assigned a name, i.e. label, by this statement. The SEGMENT and ENDS directive must bracket each logical segment of a program. In some cases, the segment may be assigned a type like PUBLIC (it can be used by other modules of the program while linking) or GLOBAL (can be accessed by any other modules). The program structure given below explains the use of the SEGMENT directive

```
EXE.CODE SEGMENT GLOBAL
```

```
EXE.CODE ENDS
```

SHORT : The SHORT operator indicates to the assembler that only one byte is required to code the displacement for a jump (i.e. displacement is within -128 to +127 bytes from the address of the byte next to the jump opcode). This method of specifying the jump address saves the memory. Otherwise, the assembler may reserve two bytes for the displacement. The syntax of the statement is as given below

```
JMP SHORT LABEL
```

MACROS in Microprocessor: Macro is a group of instructions. The Macros in Microprocessor assembler generates the code in the program each time where the macro is 'called'. Macros can be defined by MACRO and ENDM assembler directives. Creating macro is very similar to creating a new opcode that can be used in the program. It is important to note that macro sequences execute faster than procedures because there are no CALL and RET instructions to execute. The assembler places the macro instructions in the program each time when it is invoked. This procedure is known as Macro expansion. Comparison of Procedure and Macro:

Procedure	Macro
1. Accessed by CALL and RET instruction during program execution.	1. Accessed during assembly with name given to macro when defined.
2. Machine code for instructions is put only once in the memory.	2. Machine code is generated for instructions each time when macro is called.
3. With procedures less memory is required.	3. With macros more memory is required.
4. Parameters can be passed in registers, memory locations, or stack.	4. Parameters passed as part of statement which calls macro.

PROCEDURES Defining a Procedure Informally, we can define a procedure as a named block of statements that ends in a return statement.

A procedure is declared using the PROC and ENDP directives. It must be assigned a name (a valid identifier). A procedure must be written in the end of the code segment (before end directive), and it can't receive any parameter. When you create a procedure other than your program's startup procedure, end it with a RET instruction. RET forces the CPU to return to the location from where the procedure was called. A procedure is defined using the PROC and ENDP directives. The syntax is:

```
procedurename PROC
```

```
. ret procedurename ENDP
```

Example: Sum of Three Integers Let's create a procedure named SumOf that calculates the sum of three 16-bit integers. We will assume that relevant integers are assigned to AX, BX, and CX before the procedure is called. The procedure returns the sum in AX: SumOf PROC add ax,bx add ax,cx ret SumOf ENDP Invoking procedures A procedure is called (invoked) by inserting its name in the program Preceded by (call). The syntax for calling a procedure is: Call procedurename

PROGRAM :

8086 ASSEMBLY PROGRAM TO SEARCH AN ELEMENT IN AN ARRAY

```
DATA SEGMENT
STRING1 DB 11H,22H,33H,44H,55H
MSG1 DB "FOUND$"
MSG2 DB "NOT FOUND$"
SE DB 33H
DATA ENDS
```

```
PRINT MACRO MSG
MOV AH, 09H
LEA DX, MSG
INT 21H
INT 3
ENDM
```

```
CODE SEGMENT
ASSUME CS:CODE, DS:DATA
START:
MOV AX, DATA
MOV DS, AX
MOV AL, SE
LEA SI, STRING1
MOV CX, 05H
```

```
UP:
MOV BL, [SI]
CMP AL, BL
JZ FO
INC SI
DEC CX
JNZ UP
PRINT MSG2
JMP END1
```

```
FO:
PRINT MSG1
```

```
END1:
INT 3
CODE ENDS
END START
```

8086 ASSEMBLY PROGRAM TO SORT NUMBERS IN ASCENDING ORDER

```
DATA SEGMENT
STRING1 DB 99H,12H,56H,45H,36H
DATA ENDS
```

```
CODE SEGMENT
ASSUME CS:CODE,DS:DATA
START: MOV AX,DATA
MOV DS,AX
```

```
MOV CH,04H
```

```
UP2: MOV CL,04H
LEA SI,STRING1
```

```
UP1: MOV AL,[SI]
MOV BL,[SI+1]
CMP AL,BL
JC DOWN
MOV DL,[SI+1]
XCHG [SI],DL
MOV [SI+1],DL
```

```
DOWN: INC SI
DEC CL
JNZ UP1
DEC CH
JNZ UP2
```

```
INT 3
CODE ENDS
END START
```

8086 ASSEMBLY PROGRAM TO SORT NUMBERS IN descending ORDER

```
DATA SEGMENT
STRING1 DB 99H,12H,56H,45H,36H
DATA ENDS
```

```
CODE SEGMENT
ASSUME CS:CODE,DS:DATA
START: MOV AX,DATA
MOV DS,AX
```

```
MOV CH,04H
```

```
UP2: MOV CL,04H
LEA SI,STRING1
```

```
UP1: MOV AL,[SI]
MOV BL,[SI+1]
CMP AL,BL
JNC DOWN
```

```
MOV DL,[SI+1]
XCHG [SI],DL
MOV [SI+1],DL
```

```
DOWN: INC SI
DEC CL
JNZ UP1
DEC CH
JNZ UP2
```

```
INT 3
CODE ENDS
END START
```

8086 ASSEMBLY PROGRAM FOR ADDITION OF TWO 8 BIT NUMBERS

```
data segment
a db 09h
b db 02h
c dw ?
data ends
```

```
code segment
assume cs:code,ds:data
start:
mov ax,data
mov ds,ax
mov al,a
mov bl,b
add al,bl
mov c,ax
int 3
code ends
end start
```

8086 ASSEMBLY PROGRAM FOR ADDITION OF TWO 16 BIT NUMBERS

```
data segment
a dw 0149h
b dw 0172h
c dw ?
data ends
```

```
code segment
assume cs:code,ds:data
start:
mov ax,data
mov ds,ax
mov ax,a
mov bx,b
add ax,bx
```

```
mov c,ax
int 3
code ends
end start
```

8086 ASSEMBLY PROGRAM TO FIND LARGEST NUMBER FROM GIVEN NUMBERS

```
data segment
STRING1 DB 08h,14h,05h,0Fh,09h
res db ?
data ends
```

```
code segment
assume cs:code, ds:data
start: mov ax, data
mov ds, ax
mov cx, 05h
```

```
mov bl, 00h
LEA SI, STRING1
up:
mov al, [SI]
cmp al, bl
jl nxt
mov bl, al
nxt:
inc si
dec cx
jnz up
```

```
mov res,bl
int 3
code ends
end start
```

8086 ASSEMBLY PROGRAM TO FIND SMALLEST NUMBER FROM GIVEN NUMBERS

```
data segment
STRING1 DB 08h,14h,05h,0Fh,09h
res db ?
data ends
```

```
code segment
assume cs:code, ds:data
start: mov ax, data
mov ds, ax
mov cx, 05h
```

```
mov bl, 79h
LEA SI, STRING1
up:
mov al, [SI]
cmp al, bl
jge nxt
mov bl, al
nxt:
inc si
dec cx
jnz up
```

```
mov res,bl
```

```

int 3
code ends
end start

```

8086 ASSEMBLY PROGRAM FOR MULTIPLICATION OF TWO 8 BIT NUMBERS

```

data segment
a db 09h
b db 02h
c dw ?
data ends

code segment
assume cs:code, ds:data
start:
mov ax,data
mov ds,ax
mov ax,0000h
mov bx,0000h
mov al,a
mov bl,b
mul b
mov c,ax
int 3
code ends
end start

```

8086 ASSEMBLY PROGRAM FOR DIVISION OF TWO 8 BIT NUMBERS

```

data segment
a db 28h
b db 02h
c dw ?
data ends

code segment
assume cs:code, ds:data
start:
mov ax,data
mov ds,ax
mov ax,0000h
mov bx,0000h
mov al,a
mov bl,b
div b
mov c,ax
int 3
code ends
end start

```

8086 ASSEMBLY PROGRAM FOR SUBTRACTION OF TWO 8 BIT NUMBERS

ASSUME CS: CODE, DS: DATA

DATA SEGMENT

NUM1 DW 05H

```

NUM2 DW 1FH

RES DW 00H

DATA ENDS

CODE SEGMENT

START: MOV AX, DATA

MOV DS, AX

MOV AX, NUM1

MOV BX, NUM2

SUB AX, BX

MOV RES, AX

INT 3H

CODE ENDS

END START

```

16-Bit Multiplication

```

ASSUME CS: CODE, DS: DATA
DATA SEGMENT
NUM1          DW      08H
NUM2          DW      02H
RES1          DW      ?
RES2          DW      ?
DATA ENDS
CODE SEGMENT
START: MOV AX, DATA
MOV DS, AX
MOV AX, NUM1
MOV BX, NUM2
MUL BX
MOV RES1, AX
MOV RES2, DX
INT 3H
CODE ENDS
END START

```

Input:

Output:

Program No: 2(h)

16-Bit Division

```

ASSUME CS: CODE, DS:
DATA
DATA SEGMENT
NUM1          DW      00H

```

```
NUM2          DW      00H
RES1          DW      00H
RES2          DW      00H
DATA ENDS
CODE SEGMENT
START: MOV AX, DATA
MOV DS, AX
MOV AX, NUM1
MOV BX, NUM2
DIV BX
MOV RES1, AX
MOV RES2, DX
INT 3H
CODE ENDS
END START
```

Input:

Output:
0004:04h

Addition of Multi precision numbers

```
. MODEL SMALL
. DATA
. CODE
MOV AX, 0F000h
MOV BX, 1000h
ADD AX, BX
MOV CX, AX
MOV AX, 5678h
MOV BX, 1234h
ADC AX, BX
MOV DX, AX
INT 03H
END
```

Subtraction of Multi precision numbers

```
. MODEL SMALL
. DATA
. CODE
MOV AX, 0111h
MOV BX, 1000h
SUB AX, BX
MOV CX, AX
MOV AX, 5678h
MOV BX, 1234h
SBB AX, BX
MOV DX, AX
INT 03h
END
```

ALPs (8086) for Multiplication and Division

```
. MODEL SMALL
. DATA
. CODE
MOV AX, 1234h
MOV BX, 1234h
MUL BX
INT 03h
END
. MODEL SMALL
. DATA
. CODE
MOV AX, 0009H; INPUT1
MOV BL, 02H; INPUT2
DIV BL
```

INT 03h

END

Square of a given number

```
. MODEL SMALL
. DATA
NUM1 DB 03h
. CODE
MOV AX, @DATA
MOV DS, AX
MOV AL, NUM1
MUL AL
INT 03h
END
```

Cube of a given number

```
. MODEL SMALL
. DATA
NUM1 DB 02h
. CODE
MOV AX, @DATA
MOV DS, AX
MOV CL, NUM1
MOV AL, CL
MUL AL
MUL CL
INT 03h
END
```

Factorial of a given number

```
. MODEL SMALL
. DATA
NUM1 DB 04h
. CODE
MOV AX, @DATA
MOV DS, AX
MOV CL, NUM1
JCXZ A2
MOV AX, 0001H
MOV DX, 0000H
A3: MUL CL
DEC CL
JNZ A3
JMP A4
A2: MOV AL, 01H
A4: INT 03h
END
```

Positive or negative number

```
. MODEL SMALL
. DATA
NUM1 DB -12H
```

```
MSG DB „Given number is a positive number “,  
„$“
```

```
MSG1 DB „Given number is a negative number  
“, „$“
```

```
. CODE
```

```
MOV AX, @DATA
```

```
MOV DS, AX
```

```
MOV CL, NUM1
```

```
MOV AL, CL
```

```
ROL AL, 1
```

```
JC A2
```

```
MOV AH, 09H
```

```
LEA DX, MSG
```

```
INT 21H
```

```
JMP A3
```

```
A2: MOV AH, 09H
```

```
LEA DX, MSG1
```

```
INT 21H
```

```
A3: INT 03H
```

Even or odd number

```
. MODEL SMALL
```

```
. DATA
```

```
NUM1 DB 08H
```

```
MSG DB „Given data is an even number “, „$“
```

```
MSG1 DB „Given data is odd number “, „$“
```

```
. CODE
```

```
MOV AX, @DATA
```

```
MOV DS, AX
```

```
MOV CL, NUM1
```

```
MOV AX, CL
```

```
MOV BL, 02H
```

```
DIV BL
```

```
CMP AH, 00H
```

```
JZ A2
```

```
MOV AH, 09H
```

```
LEA DX, MSG1
```

```
INT 21H
```

```
JMP A3
```

```
A2: MOV AH, 09H
```

```
LEA DX, MSG
```

```
INT 21H
```

```
A3: INT 03H
```

Logical ones in a given data

```
. MODEL SMALL
```

```
. DATA
```

```
NUM1 DB 64H
```

```
. CODE
```

```
MOV AX, @DATA
```

```
MOV DS, AX
```

```
MOV BL, 00H
```

```
MOV CL, 08H
```

```
MOV DL, NUM1
```

```
MOV AL, DL
```

```
A3: ROL AL, 1
```

```
JC A2
```

```
A5: DEC CL
```

```
JNZ A3
```

```
JMP A4
```

```
A2: INC BL
```

```
JMP A5
```

```
A4: INT 03H
```

Logical zeroes in a given data

```
. MODEL SMALL
```

```
. DATA
```

```
NUM1 DB 64H
```

```
. CODE
```

```
MOV AX, @DATA
```

```
MOV DS, AX
```

```
MOV BL, 00H
```

```
MOV CL, 08H
```

```
MOV DL, NUM1
```

```
MOV AL, DL
```

```
A3: ROL AL, 1
```

```
JNC A2
```

```
A5: DEC CL
```

```
JNZ A3
```

```
JMP A4
```

```
A2: INC BL
```

```
JMP A5
```

```
A4: INT 03H
```

Largest number in a given array

```
. MODEL SMALL
```

```
. STACK
```

```
. DATA
```

```
LIST DB 02h, 09h, 03h, 06h, 08h, 07h
```

```
. CODE
```

```
MOV AX, @DATA
```

```
MOV DS, AX
```

```
MOV SI, OFFSET LIST
```

```
MOV CL, 05h
```

```
XOR AX, AX
```

```
MOV AL, [SI]
```

```
UP: INC SI
```

```
CMP AL, [SI]
```

```
JNB GO
```

```
MOV AL, [SI]
```

GO: DEC CL

JNZ UP

INT 03h

END

Smallest number in a given array

. MODEL SMALL

. STACK

. DATA

LIST DB 02h, 09h, 03h, 06h, 08h, 07h

. CODE

MOV AX, @DATA

MOV DS, AX

MOV SI, OFFSET LIST

MOV CL, 05h

XOR AX, AX

MOV AL, [SI]

UP: INC SI

CMP AL, [SI]

JB GO

MOV AL, [SI]

GO: DEC CL

JNZ UP

INT 03h

END

Ascending order

. MODEL SMALL

. STACK

. DATA

array db 66H,07H,10H,56H,23H

count db 04H

. CODE

MOV AX, @data

MOV DS, AX

MOV DL, count

a4: MOV SI, offset array

MOV CL, DL

a3: MOV AL, [SI]

CMP AL, [SI+1]

JC a2

XCHG AL, [SI+1]

XCHG AL, [SI]

a2: INC SI

DEC CL

JNZ a3

DEC DL

JNZ a4

INT 03H

END

Descending order

. MODEL SMALL

. STACK

. DATA

array db 66H,07H,10H,56H,23H

count db 04H

. CODE

MOV AX, @data

MOV DS, AX

MOV DL, count

a4: MOV SI, offset array

MOV CL, DL

a3: MOV AL, [SI]

CMP AL, [SI+1]

JNC a2

XCHG AL, [SI+1]

XCHG AL, [SI]

a2: INC SI

DEC CL

JNZ a3

DEC DL

JNZ a4

INT 03H

END

String Length

. MODEL SMALL

. DATA

S1 LABEL BYTE

LX1 DB 30H

A2 DB?

. CODE

MOV AX, @DATA

MOV DS, AX

MOV AH, 0AH

LEA DX, S1

INT 21H

MOV AH, 02H

MOV DL, A2

OR DL, 30H

INT 21H

END

Display the given String

.MODEL SMALL

.DATA

STRING DB „INDIA“,“ \$“

.CODE

MOV AX,@DATA

MOV DS, AX

```

MOV AH, 09H
LEA
DX, STRING
INT 21H
INT 03H
END

```

String Comparison

```

.MODEL SMALL
.DATA
STRG1 DB „SVCET“,“$”
STRG2 DB „SVPET“,“$”
RES DB „STRGS ARE EQUAL“,“$”
RES1 DB „STRGS ARE NOT EQUAL“,“$”
COUNT EQU 05H
.CODE
MOV AX, @DATA
MOV DS, AX
MOV ES, AX
MOV CL, COUNT
LEA SI, STRG1
LEA DI, STRG2
CLD
REPE CMPSB
JNZ A2
MOV AH, 09H
LEA DX, RES
INT 21H
JMP A1
A2: MOV AH, 09H
LEA DX, RES1
INT 21H
A1: INT 03H
END

```

Reverse String and check for palindrome

```

.MODEL SMALL
.DATA
STG DB „LIRIL“,“$”
STG1 DB 05H DUP (?),“$”
MSG DB „given string is a PALINDROME“,“$”
MSG1 DB „given string is NOT
PALINDROME“,“$”
COUNT DB 05H
.CODE
MOV AX, @DATA
MOV DS, AX
MOV ES, AX
MOV CL, COUNT
MOV SI, OFFSET STG

```

```

MOV DI, OFFSET STG1
CLD
ADD SI, 04h
A1: MOVSB
DEC SI
DEC SI
DEC CL
JNZ A1
MOV CL, COUNT
MOV AL, CL
MOV BL, 02H
DIV BL
MOV CL, AL
MOV SI, OFFSET STG
MOV DI, OFFSET STG1
REPE CMPSB
JNE A2
MOV AH, 09H
LEA DX, MSG
INT 21H
JMP A3
A2: MOV AH, 09H
LEA DX, MSG1
INT 21H
MOV AH, 09H
LEA DX, STG1
INT 21H
A3: INT 03H
END

```

Addition of Multi precision numbers

```
. MODEL SMALL
. DATA
. CODE
MOV AX, 0F000h
MOV BX, 1000h
ADD AX, BX
MOV CX, AX
MOV AX, 5678h
MOV BX, 1234h
ADC AX, BX
MOV DX, AX
INT 03H
END
```

Subtraction of Multi precision numbers

```
. MODEL SMALL
. DATA
. CODE
MOV AX, 0111h
MOV BX, 1000h
SUB AX, BX
MOV CX, AX
MOV AX, 5678h
MOV BX, 1234h
SBB AX, BX
MOV DX, AX
INT 03h
END
```

ALPs (8086) for Multiplication and Division

```
. MODEL SMALL
. DATA
. CODE
MOV AX, 1234h
MOV BX, 1234h
MUL BX
INT 03h
END
. MODEL SMALL
. DATA
. CODE
MOV AX, 0009H; INPUT1
MOV BL, 02H; INPUT2
DIV BL
```

INT 03h

END

Square of a given number

```
. MODEL SMALL
. DATA
NUM1 DB 03h
. CODE
MOV AX, @DATA
MOV DS, AX
MOV AL, NUM1
MUL AL
INT 03h
END
```

Cube of a given number

```
. MODEL SMALL
. DATA
NUM1 DB 02h
. CODE
MOV AX, @DATA
MOV DS, AX
MOV CL, NUM1
MOV AL, CL
MUL AL
MUL CL
INT 03h
END
```

Factorial of a given number

```
. MODEL SMALL
. DATA
NUM1 DB 04h
. CODE
MOV AX, @DATA
MOV DS, AX
MOV CL, NUM1
JCXZ A2
MOV AX, 0001H
MOV DX, 0000H
A3: MUL CL
DEC CL
JNZ A3
JMP A4
A2: MOV AL, 01H
A4: INT 03h
END
```

Positive or negative number

```
. MODEL SMALL
. DATA
NUM1 DB -12H
```

```
MSG DB „Given number is a positive number “,  
„$“
```

```
MSG1 DB „Given number is a negative number  
“, „$“
```

```
. CODE
```

```
MOV AX, @DATA
```

```
MOV DS, AX
```

```
MOV CL, NUM1
```

```
MOV AL, CL
```

```
ROL AL, 1
```

```
JC A2
```

```
MOV AH, 09H
```

```
LEA DX, MSG
```

```
INT 21H
```

```
JMP A3
```

```
A2: MOV AH, 09H
```

```
LEA DX, MSG1
```

```
INT 21H
```

```
A3: INT 03H
```

Even or odd number

```
. MODEL SMALL
```

```
. DATA
```

```
NUM1 DB 08H
```

```
MSG DB „Given data is an even number “, „$“
```

```
MSG1 DB „Given data is odd number “, „$“
```

```
. CODE
```

```
MOV AX, @DATA
```

```
MOV DS, AX
```

```
MOV CL, NUM1
```

```
MOV AX, CL
```

```
MOV BL, 02H
```

```
DIV BL
```

```
CMP AH, 00H
```

```
JZ A2
```

```
MOV AH, 09H
```

```
LEA DX, MSG1
```

```
INT 21H
```

```
JMP A3
```

```
A2: MOV AH, 09H
```

```
LEA DX, MSG
```

```
INT 21H
```

```
A3: INT 03H
```

Logical ones in a given data

```
. MODEL SMALL
```

```
. DATA
```

```
NUM1 DB 64H
```

```
. CODE
```

```
MOV AX, @DATA
```

```
MOV DS, AX
```

```
MOV BL, 00H
```

```
MOV CL, 08H
```

```
MOV DL, NUM1
```

```
MOV AL, DL
```

```
A3: ROL AL, 1
```

```
JC A2
```

```
A5: DEC CL
```

```
JNZ A3
```

```
JMP A4
```

```
A2: INC BL
```

```
JMP A5
```

```
A4: INT 03H
```

Logical zeroes in a given data

```
. MODEL SMALL
```

```
. DATA
```

```
NUM1 DB 64H
```

```
. CODE
```

```
MOV AX, @DATA
```

```
MOV DS, AX
```

```
MOV BL, 00H
```

```
MOV CL, 08H
```

```
MOV DL, NUM1
```

```
MOV AL, DL
```

```
A3: ROL AL, 1
```

```
JNC A2
```

```
A5: DEC CL
```

```
JNZ A3
```

```
JMP A4
```

```
A2: INC BL
```

```
JMP A5
```

```
A4: INT 03H
```

Largest number in a given array

```
. MODEL SMALL
```

```
. STACK
```

```
. DATA
```

```
LIST DB 02h, 09h, 03h, 06h, 08h, 07h
```

```
. CODE
```

```
MOV AX, @DATA
```

```
MOV DS, AX
```

```
MOV SI, OFFSET LIST
```

```
MOV CL, 05h
```

```
XOR AX, AX
```

```
MOV AL, [SI]
```

```
UP: INC SI
```

```
CMP AL, [SI]
```

```
JNB GO
```

```
MOV AL, [SI]
```

GO: DEC CL

JNZ UP

INT 03h

END

Smallest number in a given array

. MODEL SMALL

. STACK

. DATA

LIST DB 02h, 09h, 03h, 06h, 08h, 07h

. CODE

MOV AX, @DATA

MOV DS, AX

MOV SI, OFFSET LIST

MOV CL, 05h

XOR AX, AX

MOV AL, [SI]

UP: INC SI

CMP AL, [SI]

JB GO

MOV AL, [SI]

GO: DEC CL

JNZ UP

INT 03h

END

Ascending order

. MODEL SMALL

. STACK

. DATA

array db 66H,07H,10H,56H,23H

count db 04H

. CODE

MOV AX, @data

MOV DS, AX

MOV DL, count

a4: MOV SI, offset array

MOV CL, DL

a3: MOV AL, [SI]

CMP AL, [SI+1]

JC a2

XCHG AL, [SI+1]

XCHG AL, [SI]

a2: INC SI

DEC CL

JNZ a3

DEC DL

JNZ a4

INT 03H

END

Descending order

. MODEL SMALL

. STACK

. DATA

array db 66H,07H,10H,56H,23H

count db 04H

. CODE

MOV AX, @data

MOV DS, AX

MOV DL, count

a4: MOV SI, offset array

MOV CL, DL

a3: MOV AL, [SI]

CMP AL, [SI+1]

JNC a2

XCHG AL, [SI+1]

XCHG AL, [SI]

a2: INC SI

DEC CL

JNZ a3

DEC DL

JNZ a4

INT 03H

END

String Length

. MODEL SMALL

. DATA

S1 LABEL BYTE

LX1 DB 30H

A2 DB?

. CODE

MOV AX, @DATA

MOV DS, AX

MOV AH, 0AH

LEA DX, S1

INT 21H

MOV AH, 02H

MOV DL, A2

OR DL, 30H

INT 21H

END

Display the given String

.MODEL SMALL

.DATA

STRING DB „INDIA“,“ \$“

.CODE

MOV AX,@DATA

MOV DS, AX

```
MOV AH, 09H
LEA
DX, STRING
INT 21H
INT 03H
END
```

String Comparison

```
. MODEL SMALL
. DATA
STRG1 DB „SVCET“,“$”
STRG2 DB „SVPET“,“$”
RES DB „STRGS ARE EQUAL“,“$”
RES1 DB „STRGS ARE NOT EQUAL“,“$”
COUNT EQU 05H
. CODE
MOV AX, @DATA
MOV DS, AX
MOV ES, AX
MOV CL, COUNT
LEA SI, STRG1
LEA DI, STRG2
CLD
REPE CMPSB
JNZ A2
MOV AH, 09H
LEA DX, RES
INT 21H
JMP A1
A2: MOV AH, 09H
LEA DX, RES1
INT 21H
A1: INT 03H
END
```

Reverse String and check for palindrome

```
.MODEL SMALL
.DATA
STG DB „LIRIL“,“$”
STG1 DB 05H DUP (?),“$”
MSG DB „given string is a PALINDROME“,“$”
MSG1 DB „given string is NOT
PALINDROME“,“$”
COUNT DB 05H
.CODE
MOV AX, @DATA
MOV DS, AX
MOV ES, AX
MOV CL, COUNT
MOV SI, OFFSET STG
```

```
MOV DI, OFFSET STG1
CLD
ADD SI, 04h
A1: MOVSB
DEC SI
DEC SI
DEC CL
JNZ A1
MOV CL, COUNT
MOV AL, CL
MOV BL, 02H
DIV BL
MOV CL, AL
MOV SI, OFFSET STG
MOV DI, OFFSET STG1
REPE CMPSB
JNE A2
MOV AH,09H
LEA DX, MSG
INT 21H
JMP A3
A2: MOV AH, 09H
LEA DX, MSG1
INT 21H
MOV AH, 09H
LEA DX, STG1
INT 21H
A3: INT 03H
END
```